

* Nginx

Nginx. База

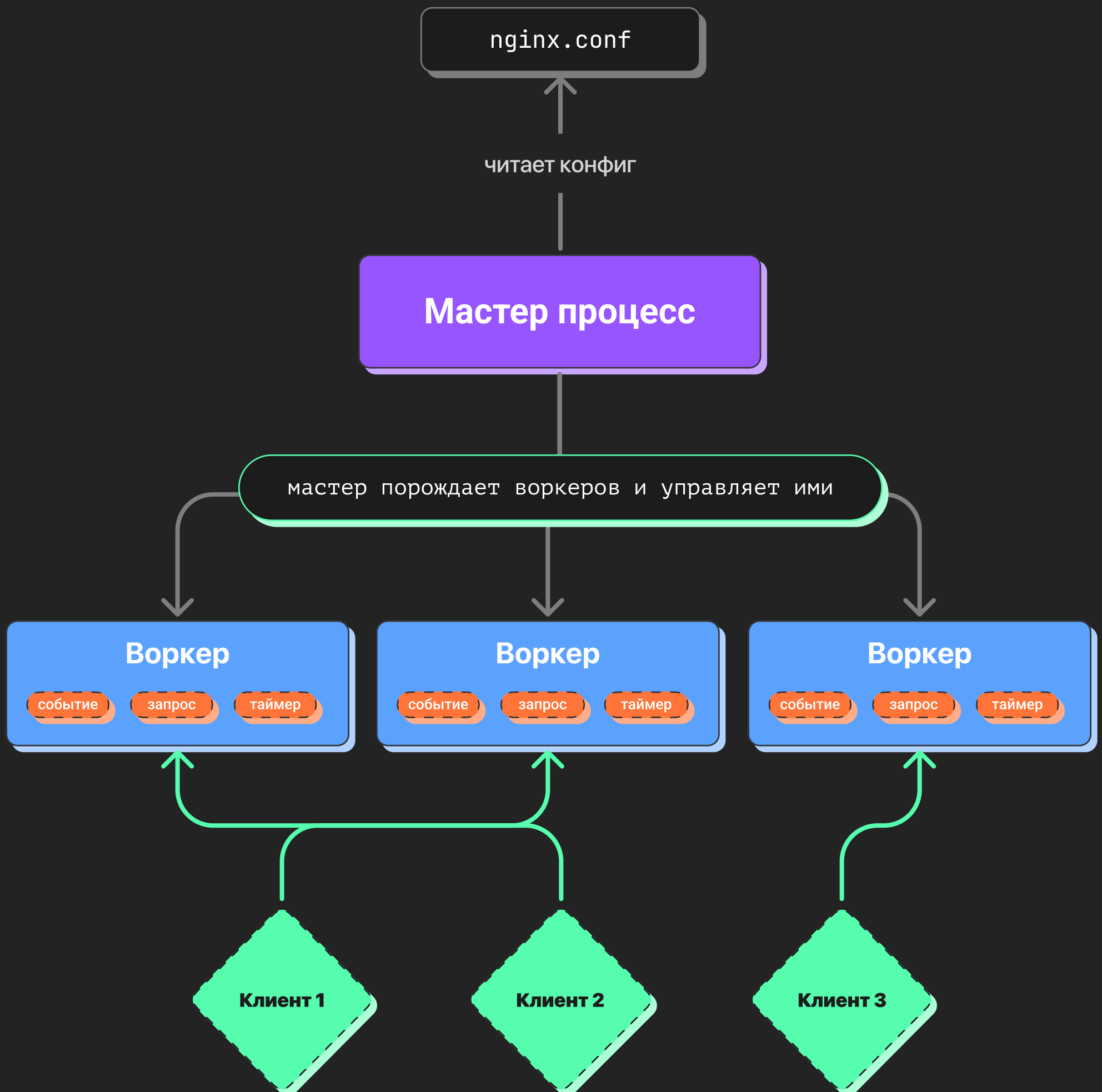
архитектура, команды, конфиг, reverse proxy



Просто Devops



1. Архитектура Nginx



2. Основные команды

```
nginx -V
```



показывает версию nginx
+ список модулей (параметры сборки)

```
nginx -t
```



проверяет синтаксис конфига на ошибки,
опечатки и т.д.

```
nginx -s reload
```



применяет конфиг без остановки сервера
клиенты не отваливаются

```
nginx -s stop
```



мягко останавливает сервер, соединения
могут отвалиться

```
systemctl restart nginx
```



сначала убивает все процессы, потом
запускает заново



запускаем только при обновлении
версии nginx, или когда он завис

3. Простой конфиг

```
user www-data;           # пользователь, под которым работает nginx
worker_processes auto;    # число воркеров (auto – по числу ядер)

events {
    worker_connections 1024; # макс. число соединений на одного воркера
}

http {
    include mime.types; # подключение типов файлов (css, html, js...)

    server {
        # блок настроек конкретного сайта
        listen 80; # порт (обычно 80 или 443)
        server_name site.com; # доменное имя сайта

        location / {
            # правило обработки запросов к корню сайта
            root /var/www; # папка, в которой лежат файлы сайта
            index index.html; # файл, который отдаем по умолчанию
        }
    }
}
```

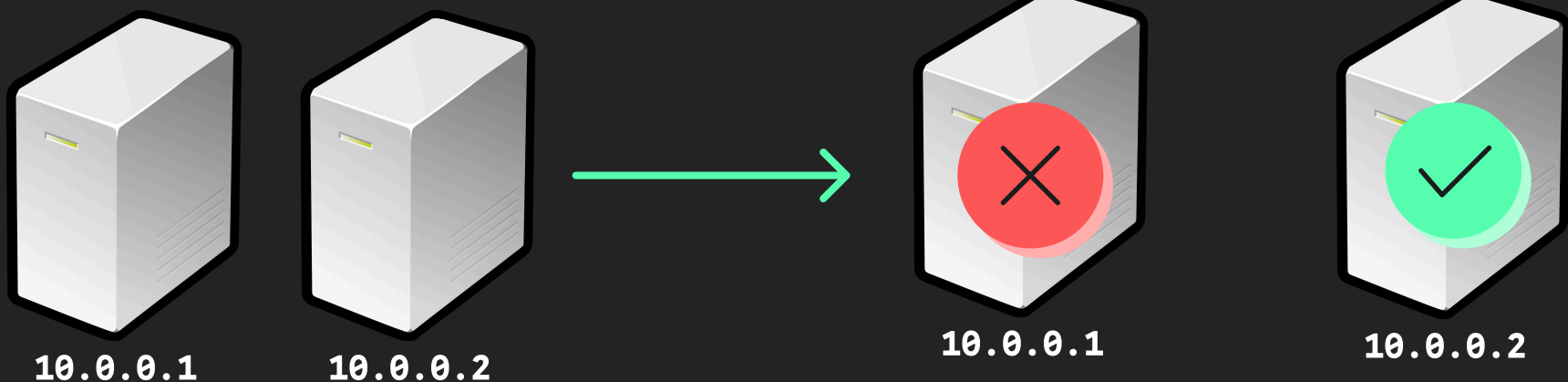
Файл обычно лежит в `/etc/nginx/nginx.conf`

Супер часто люди забывают ставить `;` в конце каждой строчки

4. Как делать балансировку

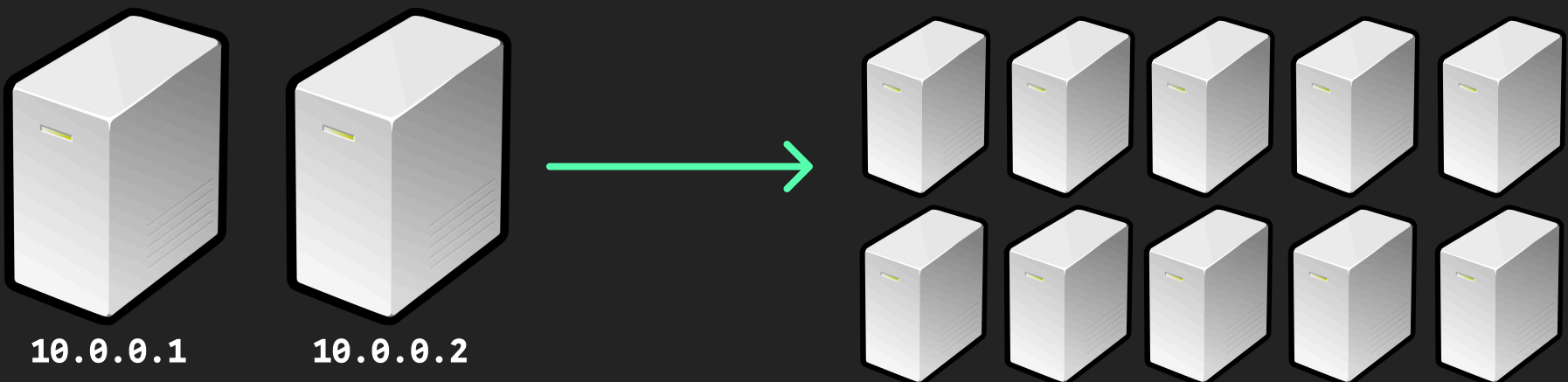
Зачем вообще нужна балансировка?

- Допустим, у нас 2 сервера



Nginx автоматом перекинет трафик на живой

- Два сервера не вывезет миллион пользователей, конечно же нужно масштабироваться



ставим 10 серваков и Nginx раскидывает нагрузку между ними

На собеседах иногда спрашивают про **Nginx**, и если спрашивают, то чаще всего про **Reverse Proxy**

Вопрос

Настраивал Nginx? Окей, как сделать Reverse Proxy (буквально с собеседа)

Ответ:

1. Создаем upstream
2. Прописываем его в proxy_pass
3. Nginx сам будет распределять запросы по очереди (Round Robin)

5. Конфиг Reverse Proxy

```
# блок upstream определяет группу бэкендов
upstream backend_api {
    server 10.0.0.1;      # первый сервер
    server 10.0.0.2;      # второй сервер
}

server {
    listen 80;
    server_name site.com;

    location / {
        # пересылаем запрос на группу upstream
        proxy_pass http://backend_api;

        # важно: передаем реальные заголовки
        proxy_set_header Host $host;
    }
}
```

Как это работает?

upstream

Сначала собираем серверы в одну группу (пул) и даем имя backend_api

proxy_pass

Когда приходит запрос, Nginx не обрабатывает его сам, а пересылает в эту группу (http://backend_api)

proxy_set_header

Вместе с запросом мы передаем реальный домен (Host), чтобы бэкенд понимал, какой сайт открывать